

Министерство образования Республики Беларусь

Учреждение образования
«Белорусский государственный университет информатики и
радиоэлектроники»

Кафедра ЭВМ

Лабораторная работа № 5
«Последовательный интерфейс SPI. ЖКИ. Акселерометр»
Вариант №9

Выполнил:
Студент группы 250501 Снитко Д.А.

Проверил:
ассистент каф. ЭВМ _____ Шеменков В.В.

Минск 2025

1. Цель работы

Цель: Изучить принципы организации последовательного интерфейса SPI и подключения устройств на его основе на базе микроконтроллера MSP430F5529.

Задача: Написать программу с использованием интерфейса SPI, ЖКИ и акселерометра в соответствии с заданием.

2. Исходные данные

Для выполнения работы используется плата MSP-EXP430F5529 и интегрированная среда разработки Code Composer Studio.

Необходимо выполнить задание варианта № 9.

Место индикации	Ориентация (поворот) текста	Зеркальное отражение	Ось измерений акселерометра
Правый верхний угол	90°	Вертикально, только символы	Y

3. Выполнение работы

3.1 Работа платы

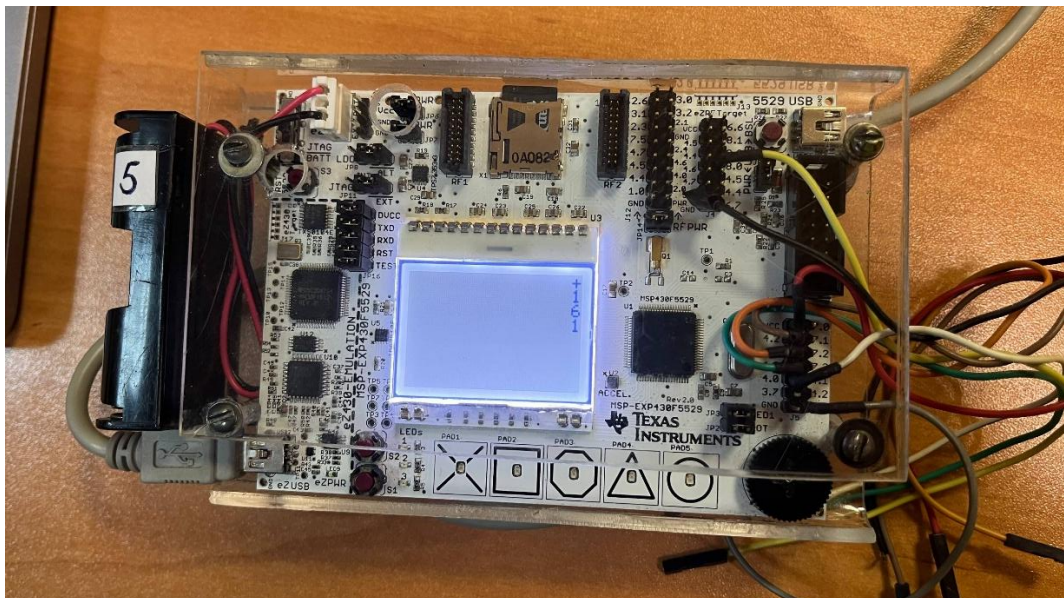


Рисунок 3.1.1 - Отображение значений акселерометра на дисплее

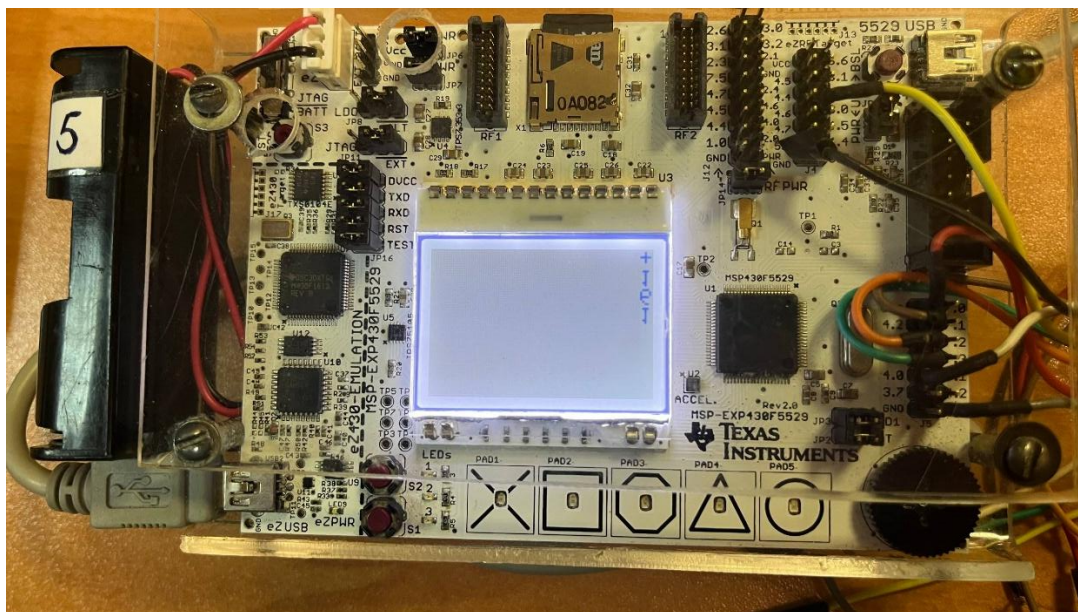


Рисунок 3.1.2 - Отображение значений акселерометра после нажатии кнопки и зеркального отражения

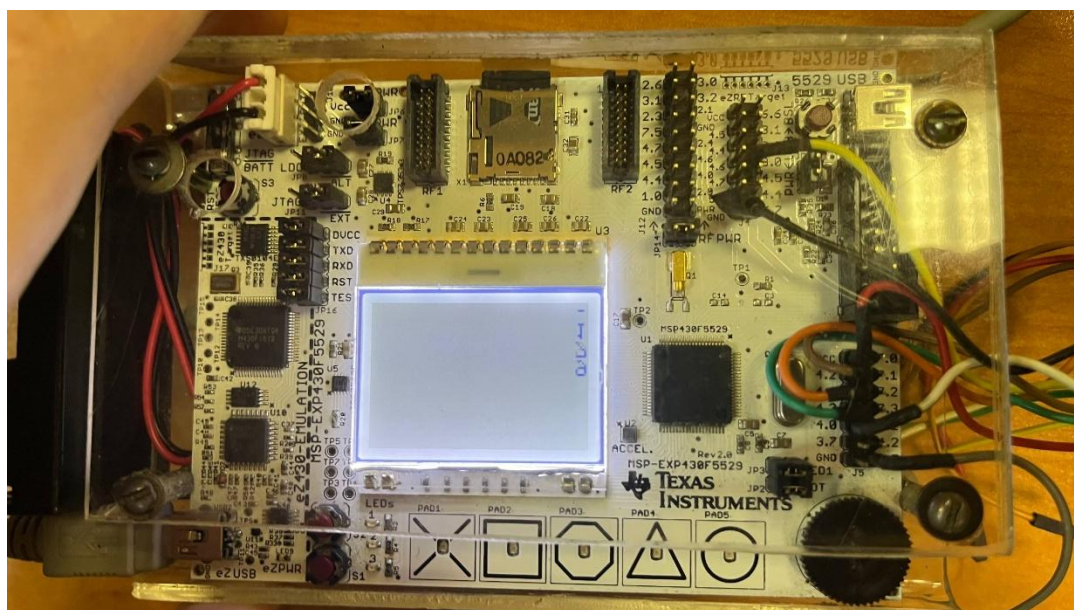


Рисунок 3.1.3 - Отображение значений акселерометра при наклоне по оси Y

3.2 Временные диаграммы SPI

На рисунке желтым показан сигнал CLK, а синим – SIMO.

Считывание данных происходит про фронту CLK. Старшие биты передаются впереди.

На рисунке 3.2.1 показана команда установки нормального отображения символов.

На рисунке 3.2.2 показана команда установки зеркального отображения символов.

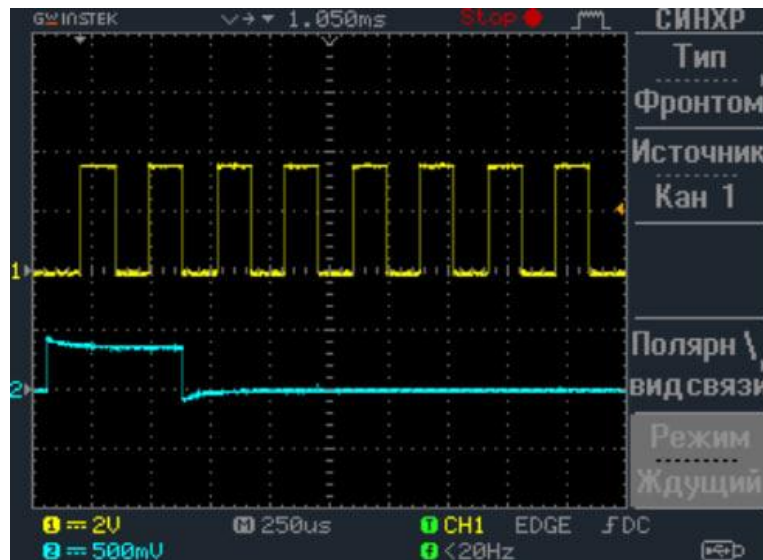


Рисунок 3.2.1 - Временная диаграмма команды установки нормального отображения символов

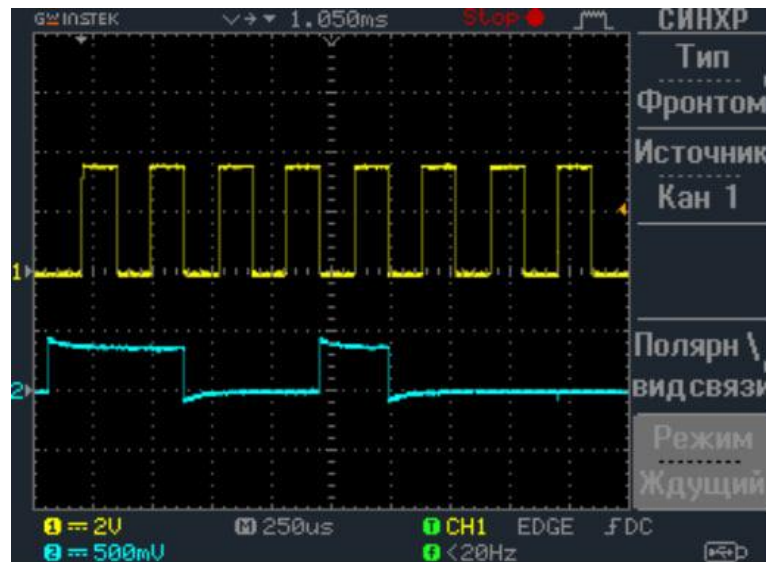


Рисунок 3.2.2 - Временная диаграмма команды установки зеркального отображения символов

3.1. Код программы

```
#include <msp430.h>

// --- КОНСТАНТЫ И ОПРЕДЕЛЕНИЯ ---
#define MCLK 25000000
#define TICKSPERUS (MCLK / 1000000)
#define DOGS102x6_X_SIZE 102
#define DOGS102x6_Y_SIZE 64
```

```

#define ACCEL_CS BIT5
#define ACCEL_PWR BIT6
#define ACCEL_SIMO BIT3
#define ACCEL_SOMI BIT4
#define ACCEL_SCK BIT7
#define ACCEL_INT BIT5
#define ACCEL_OUT P3OUT
#define ACCEL_DIR P3DIR
#define ACCEL_SEL P3SEL
#define ACCEL_SCK_SEL P2SEL
#define ACCEL_INT_DIR P2DIR
#define ACCEL_INT_IN P2IN
#define ACCEL_INT_IES P2IES
#define ACCEL_INT_IFG P2IFG
#define ACCEL_INT_IE P2IE
#define REVID 0x01
#define CTRL 0x02
#define MODE_400 0x04
#define DOUTY 0x07
#define G_RANGE_2 0x80
#define I2C_DIS 0x10
#define CD BIT6
#define CS BIT4
#define RST BIT7
#define BACKLT BIT6
#define SPI_SIMO BIT1
#define SPI_CLK BIT3
#define CD_RST_DIR P5DIR
#define CD_RST_OUT P5OUT
#define CS_BACKLT_DIR P7DIR
#define CS_BACKLT_OUT P7OUT
#define CS_BACKLT_SEL P7SEL
#define SPI_SEL P4SEL
#define SPI_DIR P4DIR

#define SET_COLUMN_ADDRESS_MSB 0x10
#define SET_COLUMN_ADDRESS_LSB 0x00
#define SET_PAGE_ADDRESS 0xB0
#define SET_POWER_CONTROL 0x2F
#define SET_SCROLL_LINE 0x40
#define SET_VLCD_RESISTOR_RATIO 0x27
#define SET_ELECTRONIC_VOLUME_MSB 0x81
#define SET_ELECTRONIC_VOLUME_LSB 0x0F
#define SET_ALL_PIXEL_ON 0xA4

```

```

#define SET_INVERSE_DISPLAY_OFF 0xA6
#define SET_DISPLAY_ENABLE 0xAF
#define SET_SEG_DIRECTION 0xA1
#define SET_COM_DIRECTION 0xC8
#define SYSTEM_RESET 0xE2
#define SET_LCD_BIAS_RATIO 0xA2
#define SET_ADV_PROGRAM_CONTROL0_MSB 0xFA
#define SET_ADV_PROGRAM_CONTROL0_LSB 0x90

// --- ГЛОБАЛЬНЫЕ ПЕРЕМЕННЫЕ ---
char Cma3000_yAccel;
int num_y = 0;
unsigned char dogs102x6Memory[818];
unsigned char backlight = 0;

// Флаг для зеркального отображения: 1 = нормальное, 0 =
зеркальное по горизонтали
int mirror_mode = 1;

int MAPPING_VALUES[] = { 1142, 571, 286, 143, 71, 36, 18 };
unsigned char BITx[] = { BIT6, BIT5, BIT4, BIT3, BIT2, BIT1,
BIT0 };

static const unsigned char FONT6x8[] = {
    0x00, 0x00, 0x00, 0x00, 0x00, 0x00, // (char 32) ' '
    0x00, 0x00, 0xFA, 0x00, 0x00, 0x00, // (char 33) '!'
    0x00, 0xE0, 0x00, 0xE0, 0x00, 0x00, // (char 34) '"'
    0x28, 0xFE, 0x28, 0xFE, 0x28, 0x00, // (char 35) '#'
    0x24, 0x54, 0xFE, 0x54, 0x48, 0x00, // (char 36) '$'
    0xC4, 0xC8, 0x10, 0x26, 0x46, 0x00, // (char 37) '%'
    0x6C, 0x92, 0x6A, 0x04, 0x0A, 0x00, // (char 38) '&'
    0x00, 0x10, 0xE0, 0xC0, 0x00, 0x00, // (char 39) '''
    0x00, 0x38, 0x44, 0x82, 0x00, 0x00, // (char 40) '('
    0x00, 0x82, 0x44, 0x38, 0x00, 0x00, // (char 41) ')'
    0x54, 0x38, 0xFE, 0x38, 0x54, 0x00, // (char 42) '*'
    0x08, 0x08, 0x3E, 0x08, 0x08, 0x00, // (char 43) '+'
    0x00, 0x02, 0x1C, 0x18, 0x00, 0x00, // (char 44) ','
    0x00, 0x08, 0x08, 0x08, 0x00, 0x00, // (char 45) '-'
    0x00, 0x00, 0x06, 0x06, 0x00, 0x00, // (char 46) '.'
    0x04, 0x08, 0x10, 0x20, 0x40, 0x00, // (char 47) '/'
    0x7C, 0x8A, 0x92, 0xA2, 0x7C, 0x00, // (char 48) '0'
    0x00, 0x42, 0xFE, 0x02, 0x00, 0x00, // (char 49) '1'
    0x42, 0x86, 0x8A, 0x92, 0x62, 0x00, // (char 50) '2'
    0x84, 0x82, 0x92, 0xB2, 0xCC, 0x00, // (char 51) '3'

```

```

        0x18, 0x28, 0x48, 0xFE, 0x08, 0x00, // (char 52) '4'
        0xE4, 0xA2, 0xA2, 0xA2, 0x9C, 0x00, // (char 53) '5'
        0x3C, 0x52, 0x92, 0x92, 0x0C, 0x00, // (char 54) '6'
        0x82, 0x84, 0x88, 0x90, 0xE0, 0x00, // (char 55) '7'
        0x6C, 0x92, 0x92, 0x92, 0x6C, 0x00, // (char 56) '8'
        0x60, 0x92, 0x92, 0x94, 0x78, 0x00 // (char 57) '9'
    };

    unsigned char Dogs102x6_initMacro[] = {
        SET_SCROLL_LINE,    SET_SEG_DIRECTION,    SET_COM_DIRECTION,
SET_ALL_PIXEL_ON,
        SET_INVERSE_DISPLAY_OFF,                SET_LCD_BIAS_RATIO,
SET_POWER_CONTROL,
        SET_VLCD_RESISTOR_RATIO,                SET_ELECTRONIC_VOLUME_MSB,
SET_ELECTRONIC_VOLUME_LSB,
        SET_ADV_PROGRAM_CONTROL0_MSB,
SET_ADV_PROGRAM_CONTROL0_LSB, SET_DISPLAY_ENABLE,
        SET_PAGE_ADDRESS,                SET_COLUMN_ADDRESS_MSB,
SET_COLUMN_ADDRESS_LSB
    };

    // --- ПРОТОТИПЫ ФУНКЦИЙ ---
    void Dogs102x6_writeCommand(unsigned char *sCmd, unsigned
char i);
    void Dogs102x6_setAddress(unsigned char pa, unsigned char
ca);
    void Dogs102x6_refresh(void);
    void Dogs102x6_init(void);
    void Dogs102x6_backlightInit(void);
    void Dogs102x6_setBacklight(unsigned char brightness);
    void setPixel(int x, int y, int color);
    void drawCharMinus90(int x_start, int y_start, unsigned int
f, int mirror);
    void ShowNumber_Minus90(int num);
    int parseProjectionByte(unsigned char projectionByte);
    char Cma3000_readRegister(char Address);
    char Cma3000_writeRegister(unsigned char Address, char
accelData);
    void Cma3000_init(void);
    void SetupButtons(void);
    int abs_val(int num);

    // --- ФУНКЦИИ ДЛЯ РАБОТЫ С ДИСПЛЕЕМ ---

```

```

void setPixel(int x, int y, int color) {
    if (x < 0 || x >= DOGS102x6_X_SIZE || y < 0 || y >=
DOGS102x6_Y_SIZE) return;
    unsigned char page = y / 8;
    unsigned char bit_pos = y % 8;
    unsigned int mem_addr = 2 + (page * 102) + x;
    if (color) dogs102x6Memory[mem_addr] |= (1 << bit_pos);
    else dogs102x6Memory[mem_addr] &= ~(1 << bit_pos);
}

void drawCharMinus90(int x_start, int y_start, unsigned int
f, int mirror) {
    if (f < 32 || f > 57) f = '0'; // Ограничение символов 0-
9 и знаки
    const unsigned char* font_char = &FONT6x8[(f - 32) * 6];
    int col_orig, row_orig;
    for (col_orig = 0; col_orig < 6; col_orig++) {
        unsigned char font_byte = font_char[col_orig];
        for (row_orig = 0; row_orig < 8; row_orig++) {
            if ((font_byte >> row_orig) & 1) {
                // Постоянное горизонтальное зеркалирование
                int dest_x = x_start - (5 - col_orig);
                // Переключаемое вертикальное зеркалирование
                int dest_y = y_start + (mirror ? row_orig :
(7 - row_orig));
                setPixel(dest_x, dest_y, 1);
            }
        }
    }
}

void ShowNumber_Minus90(int num) {
    unsigned int i;
    for (i = 2; i < 818; i++) dogs102x6Memory[i] = 0x00;

    int length = 0;
    long temp = abs_val(num);
    if (temp == 0) length = 1;
    else { long n = temp; while (n > 0) { n /= 10; length++;
} }

    int x_start = DOGS102x6_X_SIZE - 1; // Правый край
    int y_start = DOGS102x6_Y_SIZE - 8; // Нижний край

```



```

        // Рисуем знак
        drawCharMinus90(x_start, y_start, (num < 0) ? '-' : '+',
mirror_mode);
        y_start -= 8; // Шаг вверх

        // Рисуем цифры
        if (temp == 0) {
            drawCharMinus90(x_start, y_start, '0', mirror_mode);
            y_start -= 8;
        } else {
            long divisor = 1;
            for (i = 0; i < length - 1; i++) divisor *= 10;
            while (divisor > 0) {
                drawCharMinus90(x_start, y_start, (temp /
divisor) + '0', mirror_mode);
                y_start -= 8;
                temp %= divisor;
                divisor /= 10;
            }
        }

        Dogs102x6_refresh();
    }

void Dogs102x6_refresh(void) {
    unsigned int gie = __get_SR_register() & GIE;
    __disable_interrupt();
    unsigned char p;
    for (p = 0; p < 8; p++) {
        Dogs102x6_setAddress(p, 0);
        P7OUT &= ~CS; P5OUT |= CD;
        unsigned char i;
        for (i = 0; i < 102; i++) {
            while (!(UCB1IFG & UCTXIFG));
            UCB1TXBUF = dogs102x6Memory[2 + (p * 102) + i];
        }
        while (UCB1STAT & UCBUSY);
        UCB1RXBUF;
        P7OUT |= CS;
    }
    __bis_SR_register(gie);
}

```

```

void Dogs102x6_init(void) {
    CD_RST_DIR |= RST; CD_RST_OUT &= ~RST; CD_RST_OUT |= RST;
    CS_BACKLT_DIR |= CS; CS_BACKLT_OUT &= ~CS;
    CD_RST_DIR |= CD; CD_RST_OUT &= ~CD;
    SPI_SEL  |= SPI_SIMO  | SPI_CLK; SPI_DIR  |= SPI_SIMO  |
SPI_CLK;
    UCB1CTL1 |= UCSWRST;
    UCB1CTL0 = UCCKPH + UCMSB + UCMST + UCMODE_0 + UCSYNC;
    UCB1CTL1 = UCSSEL_2 + UCSWRST;
    UCB1BR0 = 0x02; UCB1BR1 = 0;
    UCB1CTL1 &= ~UCSWRST; UCB1IFG &= ~UCRXIFG;
    Dogs102x6_writeCommand(Dogs102x6_initMacro,
sizeof(Dogs102x6_initMacro));
    CS_BACKLT_OUT |= CS;
    dogs102x6Memory[0] = 102; dogs102x6Memory[1] = 8;
}

void Dogs102x6_writeCommand(unsigned char *sCmd, unsigned
char i) {
    unsigned int gie = __get_SR_register() & GIE;
    __disable_interrupt();
    P7OUT &= ~CS; P5OUT &= ~CD;
    while (i--) {
        while (!(UCB1IFG & UCTXIFG));
        UCB1TXBUF = *sCmd++;
    }
    while (UCB1STAT & UCBUSY);
    UCB1RXBUF;
    P7OUT |= CS;
    __bis_SR_register(gie);
}

void Dogs102x6_setAddress(unsigned char pa, unsigned char ca)
{
    unsigned char cmd[1];
    if (pa > 7) pa = 7; if (ca > 101) ca = 101;
    cmd[0] = SET_PAGE_ADDRESS + pa;
    unsigned char H = (ca & 0xF0) >> 4, L = ca & 0x0F;
    unsigned char ColumnAddress[] = { SET_COLUMN_ADDRESS_LSB
+ L, SET_COLUMN_ADDRESS_MSB + H };
    Dogs102x6_writeCommand(cmd, 1);
    Dogs102x6_writeCommand(ColumnAddress, 2);
}

```

```

void Dogs102x6_backlightInit(void) {
    CS_BACKLT_DIR    |=  BACKLT;    CS_BACKLT_OUT    |=  BACKLT;
CS_BACKLT_SEL |= BACKLT;
    TBOCCTL4 = OUTMOD_7;
    TBOCCR4 = 25; TBOCCR0 = 50;
    TBOCTL = TBSSEL_1 + MC_1;
}

void Dogs102x6_setBacklight(unsigned char brightness) {
    if (brightness > 15) brightness = 15;
    if (brightness > 0) {
        TBOCCTL4 = OUTMOD_7;
        TBOCCR4 = (TBOCCR0 * brightness) / 15;
    } else {
        TBOCCTL4 = OUTMOD_0; TBOCCR4 = 0;
    }
    backlight = brightness;
}

void Cma3000_init(void) {
    do {
        ACCEL_OUT |= ACCEL_PWR; ACCEL_DIR |= ACCEL_PWR;
        ACCEL_SEL |= ACCEL_SIMO + ACCEL_SOMI; ACCEL_SCK_SEL
|= ACCEL_SCK;
        ACCEL_INT_DIR    &=    ~ACCEL_INT;    ACCEL_INT_IES    &=
~ACCEL_INT; ACCEL_INT_IFG &= ~ACCEL_INT;
        ACCEL_OUT |= ACCEL_CS; ACCEL_DIR |= ACCEL_CS;
        UCA0CTL1 |= UCSWRST;
        UCA0CTL0 = UCMST + UCSYNC + UCCKPH + UCMSB;
        UCA0CTL1 = UCSWRST + UCSSEL_2;
        UCA0BR0 = 0x30; UCA0BR1 = 0; UCA0MCTL = 0;
        UCA0CTL1 &= ~UCSWRST;
        Cma3000_readRegister(REVID);
        __delay_cycles(50 * TICKSPERUS);
        Cma3000_writeRegister(CTRL,  G_RANGE_2  |  I2C_DIS  |
MODE_400);
        __delay_cycles(1000 * TICKSPERUS);
        ACCEL_INT_IE &= ~ACCEL_INT;
    } while (!(ACCEL_INT_IN & ACCEL_INT));
}

char  Cma3000_writeRegister(unsigned  char  Address,  char
accelData) {
    char Result;

```

```

    Address = (Address << 2) | 2;
    ACCEL_OUT &= ~ACCEL_CS;
    while (!(UCA0IFG & UCTXIFG)); UCA0TXBUF = Address;
    while (!(UCA0IFG & UCRXIFG)); UCA0RXBUF;
    while (!(UCA0IFG & UCTXIFG)); UCA0TXBUF = accelData;
    while (!(UCA0IFG & UCRXIFG)); Result = UCA0RXBUF;
    while (UCA0STAT & UCBUSY);
    ACCEL_OUT |= ACCEL_CS;
    return Result;
}

char Cma3000_readRegister(char Address) {
    char Result;
    Address <= 2;
    ACCEL_OUT &= ~ACCEL_CS;
    UCA0RXBUF;
    while (!(UCA0IFG & UCTXIFG)); UCA0TXBUF = Address;
    while (!(UCA0IFG & UCRXIFG)); Result = UCA0RXBUF;
    while (!(UCA0IFG & UCTXIFG)); UCA0TXBUF = 0;
    while (!(UCA0IFG & UCRXIFG)); Result = UCA0RXBUF;
    while (UCA0STAT & UCBUSY);
    ACCEL_OUT |= ACCEL_CS;
    return Result;
}

int parseProjectionByte(unsigned char projectionByte) {
    int i, projectionValue = 0;
    int isNegative = projectionByte & BIT7;
    for (i = 0; i < 7; i++) {
        if (isNegative) projectionValue += (BITx[i] &
projectionByte) ? 0 : MAPPING_VALUES[i];
        else projectionValue += (BITx[i] & projectionByte) ?
MAPPING_VALUES[i] : 0;
    }
    return isNegative ? -projectionValue : projectionValue;
}

int abs_val(int num) {
    return (num > 0) ? num : -num;
}

void SetupButtons(void) {
    P1DIR &= ~BIT7; P1REN |= BIT7; P1OUT |= BIT7;
    P1IE |= BIT7; P1IES |= BIT7; P1IFG &= ~BIT7;

```

```

        P2IE = 0; P2IFG = 0;
    }

    // --- ПЕРЕРЫВАНИЯ И ГЛАВНАЯ ФУНКЦИЯ ---

#pragma vector=WDT_VECTOR
__interrupt void WDTINT(void) {
    SFRIE1 &= ~WDTIE;

    num_y
    parseProjectionByte(Cma3000_readRegister(DOUTY));
    ShowNumber_Minus90(num_y);
    SFRIE1 |= WDTIE;
}

#pragma vector=PORT1_VECTOR
__interrupt void buttonS1(void) {
    __delay_cycles(50000);
    if (!(P1IN & BIT7)) {
        mirror_mode = !mirror_mode;
        ShowNumber_Minus90(num_y);
    }
    P1IFG &= ~BIT7;
}

int main(void) {
    WDTCTL = WDT_ADLY_250;
    SFRIE1 |= WDTIE;

    SetupButtons();
    __bis_SR_register(GIE);

    Cma3000_init();
    Dogs102x6_init();
    Dogs102x6_backlightInit();
    Dogs102x6_setBacklight(8);

    ShowNumber_Minus90(0);

    while (1) {
        __no_operation();
    }
    return 0;
}

```

4. Выводы

В ходе выполнения лабораторной работы удалось изучить принципы организации последовательного интерфейса SPI и подключения устройств на его основе на базе микроконтроллера MSP430F5529.